Vel Tech
Rangarajan Dr. Sagunthala
R&D Institute of Science and Technology
(Deemed to be University Estd. u/s 3 of UGC Act, 1956)

**School of Computing**

**Department of Computer Science and Engineering**

**ACADEMIC YEAR 2026–2027**

**Summer Semester**

**10212CS211 – ARTIFICIAL INTELLIGENCE AND TECHNIQUES**

**FACULTY NAME** : Dr. R. Roselin Kiruba

**SLOT** : S1 & L6

**YEAR / SEM** : III / 5th

**USE CASE NAME** : AI-Based Multi-Agent Traffic Management System

| S. NO | VTU NO | NAME | SIGNATURE |
|---|---|---|---|
| 1 | 30207 | Rounak Kumar Agarwal | R. Agarwal |
| 2 | 29887 | ADVAITA MISHRA V | Advaita |
| 3 | 29901 | Jalla Anusha | J. Anusha |
| 4 | 29913 | Marthala Manju Sree | M. Manju |
| 5 | 29916 | Bathala Divya Chaithanya | |
| 6 | 30114 | Thommandula Dheeraj | T. Dheeraj |

**FACULTY SIGNATURE**

# AI-BASED MULTI-AGENT TRAFFIC MANAGEMENT SYSTEM

Intelligent Intersection Control Using Reactive, Goal-Driven, Utility-Driven and Learning Agents

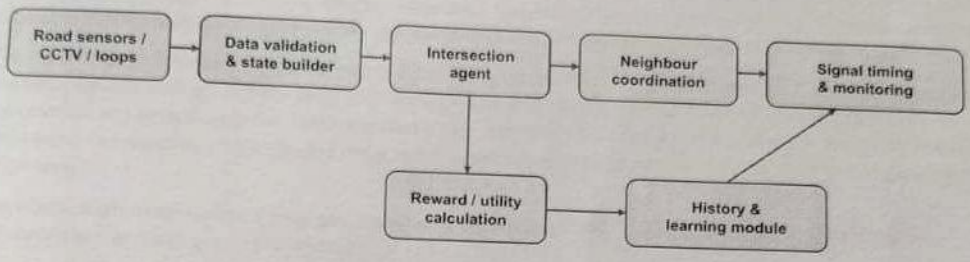



*Figure: Proposed cooperative traffic-control architecture*

**PROJECT REPORT**

Submitted in partial fulfilment of academic project requirements

Prepared by: Raunak, Advaita, Anushka, Manasa Shree, Divya, Dhanraj

Department: Computer Science and Engineering

Institution: Vel Tech Rangarajan Dr. Sagunthala R&D Institute of Science and Technology

Academic Year: 2026–2027

## Abstract

Urban traffic congestion increases travel time, fuel consumption, emissions and the delay experienced by emergency vehicles and public transport. Fixed-time signals cannot respond adequately to rapidly changing vehicle density, queue length, waiting time and traffic flow. This report proposes a multi-agent traffic management system in which each road intersection is controlled by an autonomous software agent. Agents observe local traffic, choose signal actions, exchange concise state information with neighbouring intersections and revise decisions as conditions change.

The proposed design combines reactive rules for immediate safety, goal-driven behaviour for queue and delay reduction, utility-based action selection for balancing competing approaches, and a learning component that can improve timing choices from measured outcomes. A coordination layer discourages a busy intersection from releasing traffic into a downstream road that is already congested. A bounded decision cycle selects a green phase and duration subject to minimum/maximum green times, yellow intervals, all-red clearance and pedestrian constraints.

The report includes a modular architecture, agent models, a utility function, pseudocode, an illustrative four-approach example, sample inter-agent messages, evaluation metrics and implementation guidance. Numerical values are hypothetical and demonstrate the decision process only; they are not measured traffic results or a validated deployment. Real-world use would require calibrated sensors, road-network modelling, simulation, field trials and approval by traffic engineers.



Keywords: multi-agent systems, intelligent transportation, adaptive traffic signals, utility function, coordination, reinforcement learning, queue management.


### Report objectives

- Represent each intersection as an autonomous agent with a clear state, action set and communication interface.
- Select signal phases using density, queue length, waiting time, flow and downstream conditions.
- Coordinate adjacent intersections to reduce spillback and unnecessary stops.
- Demonstrate a transparent sample decision and the expected output format.
- Define testing, safety constraints and metrics for a responsible prototype.

# Table of Contents


| Section | Page |
|---|---|
| Abstract | 2 |
| 1. Introduction | 4 |
| 2. Problem Definition and Objectives | 5 |
| 3. Background and Agent Types | 6 |
| 4. Requirements and Scope | 7 |
| 5. System Architecture | 8 |
| 6. Agent State, Actions and Communication | 9 |
| 7. Utility Function and Decision Policy | 10 |
| 8. Coordination Strategy | 11 |
| 9. Proposed Algorithm and Pseudocode | 12 |
| 10. Implementation Design | 13 |
| 11. Worked Example and Sample Input | 14 |
| 12. Sample Output and Interpretation | 15 |
| 13. Testing and Evaluation Plan | 16 |
| 14. Safety, Limitations and Deployment | 17 |
| 15. Conclusion and References | 18 |


## How to read this report

The first chapters define the congestion problem and the agent-based approach. The middle chapters describe the architecture, intersection state, utility function, coordination protocol and algorithm. The worked example illustrates how a local agent chooses a phase while considering messages from neighbours. The final chapters define evaluation measures and safety constraints. The report is a proposed system design, not a claim that a deployed traffic-control product has already been validated.

# 1. Introduction

Traffic networks are dynamic systems. Demand changes by time of day, road works alter capacity, incidents create unexpected queues and a green signal at one junction can send a platoon of vehicles toward the next junction. A fixed schedule may allocate green time to an empty approach while vehicles wait on another approach. When intersections operate independently, these local decisions can also create stop-and-go waves and queue spillback.

Artificial intelligence offers a way to make traffic-control decisions from the current state rather than relying only on a timetable. In a multi-agent system, every intersection is an agent that senses local conditions, evaluates actions and exchanges information with nearby agents. The overall network emerges from coordinated local decisions; a single central controller is not required to choose every signal phase.

## 1.1 Motivation

- Responsiveness: adapt signal timing when a queue grows or waiting time becomes excessive.
- Coordination: avoid releasing traffic into a downstream segment with limited remaining storage.
- Fairness: prevent low-volume approaches from being ignored indefinitely.
- Efficiency: reduce total delay, queue length, unnecessary stops and spillback.
- Explainability: record why a phase was selected and which constraints were applied.

## 1.2 Proposed contribution

The proposed prototype uses a hybrid agent. Reactive rules handle signal safety and immediate exceptional conditions; goal-driven logic targets queue and delay reduction; utility scoring compares competing green phases; and an optional learning module updates policy parameters using observed outcomes. Communication is limited to actionable neighbour summaries so that the system remains understandable and can tolerate delayed messages.

## 1.3 Operating assumption

The initial prototype controls a simplified network of signalised intersections with reliable phase definitions and lane-level or approach-level counts. Pedestrian calls, emergency pre-emption and legal signal timing are treated as hard constraints rather than rewards that an optimiser may trade away.

# 2. Problem Definition and Objectives

**Problem statement.** Design an AI-based multi-agent traffic management system for a metropolitan road network in which each intersection is controlled by an intelligent agent. Agents receive vehicle density, queue length, waiting time, current signal status and traffic-flow information, communicate with neighbouring intersections, and select safe signal phases and durations that respond to changing traffic conditions.

## 2.1 Inputs

| Input | Example representation | Purpose |
|---|---|---|
| Vehicle density | Vehicles per lane or occupancy percentage | Indicates demand and congestion pressure |
| Queue length | Number of vehicles or metres | Estimates storage use and likely delay |
| Waiting time | Seconds per approach; maximum and average | Identifies approaches needing service |
| Traffic flow | Vehicles/minute; turning proportions | Estimates arrivals and discharge |
| Signal status | Current phase, elapsed green, clearance state | Prevents unsafe or invalid phase changes |
| Neighbour message | Downstream queue, receiving capacity, phase estimate | Supports coordination and spillback prevention |
| Context | Time, pedestrian call, emergency request, sensor quality | Adds constraints and data reliability |

## 2.2 Expected outputs

- Chosen approach/phase and requested green duration.
- Decision explanation: queue pressure, waiting-time pressure, flow and downstream penalty.
- Agent-to-agent messages containing compact traffic-state summaries.
- Updated phase configuration and a log of the decision.

## 2.3 Objectives

The principal objective is to reduce network-wide delay without creating unsafe signal transitions or unfair starvation. Secondary objectives include lowering maximum queue length, improving throughput and emergency-route responsiveness, and limiting the amount of communication needed between intersections.

# 3. Background and Agent Types

A multi-agent system consists of multiple autonomous decision-making entities that perceive part of an environment and interact to achieve individual or shared objectives. A road intersection is a natural agent boundary because it has local sensors, a finite set of legal signal phases and neighbouring road links. The traffic network is the shared environment.

## 3.1 Agent types considered

| Agent type | Behaviour in traffic control | Strength | Limitation |
|---|---|---|---|
| Reactive | Uses immediate rules, e.g. extend green when queue is high, subject to safety bounds | Fast and predictable | Can be shortsighted |
| Goal-driven | Chooses actions that move toward targets such as lower delay or clearing a queue | Objective-oriented | May need search or planning |
| Utility-driven | Scores legal actions and selects the action with the best weighted trade-off | Balances multiple priorities | Weights need calibration |
| Learning agent | Updates policy from outcomes, e.g. reward for reduced delay and balanced service | Can adapt to recurring patterns | Needs safe training and monitoring |
| Cooperative multi-agent | Exchanges local state and considers neighbour capacity | Can reduce network spillback | Messages can be delayed or incomplete |

## 3.2 Recommended hybrid design

Use reactive logic as the safety and fallback layer, utility-driven scoring for ordinary phase selection, goal-driven checks for maximum-wait and queue-clearing objectives, and learning as an optional later enhancement. Learning must not bypass legal signal constraints. If data or communication fails, the agent falls back to an approved safe timing plan.

## 3.3 Relationship to centralised control

A central optimiser can see the full network but may become a communication bottleneck or single point of failure. Fully independent agents are resilient but may make conflicting decisions. The proposed approach is decentralised local control with bounded neighbour coordination and optional supervisory monitoring.

# 4. Requirements and Scope

## 4.1 Functional requirements

- FR1: Collect and time-stamp traffic observations for each approach.
- FR2: Validate missing, stale or implausible sensor readings and attach quality flags.
- FR3: Maintain current phase, elapsed green and legal transition state.
- FR4: Estimate queue pressure, waiting-time pressure and traffic flow.
- FR5: Generate only legally permitted candidate phases and green durations.
- FR6: Calculate utility for each candidate and select the best valid action.
- FR7: Exchange downstream congestion and receiving-capacity summaries with neighbours.
- FR8: Apply anti-starvation and maximum-wait rules.
- FR9: Log input, message context, chosen action, utility components and outcome.
- FR10: Revert to a safe fallback plan during sensor, communication or controller faults.

## 4.2 Non-functional requirements

| Requirement | Design target |
|---|---|
| Safety | Hard constraints for yellow/all-red, intergreen, pedestrian and emergency rules |
| Responsiveness | Decision within a defined control interval, e.g. every few seconds |
| Robustness | Graceful handling of stale messages, missing detectors and restarts |
| Interpretability | Record the contribution of each utility component |
| Scalability | Exchange compact messages only with directly connected neighbours |
| Auditability | Timestamp decisions, controller state, policy version and overrides |
| Security | Authenticate controller communications and reject malformed messages |

## 4.3 Scope and exclusions

The report covers a conceptual algorithm and an illustrative intersection example. It does not connect to live city controllers, guarantee travel-time improvements, or replace traffic engineering approval. The first implementation should use a microscopic traffic simulator and a small, clearly defined road network before any field test.

# 5. System Architecture

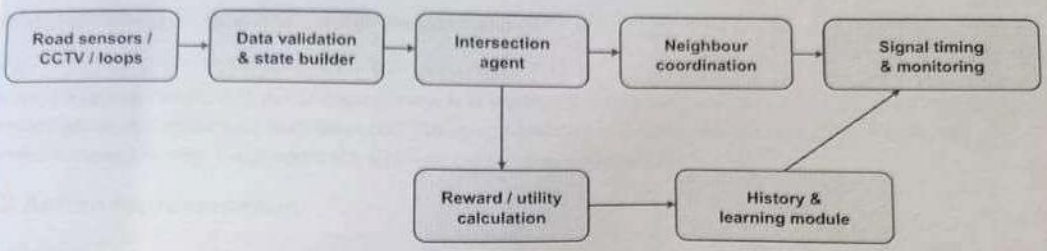



*Figure 1. High-level architecture of the proposed multi-agent traffic workflow.*

## 5.1 Module descriptions

| Module | Responsibility |
|---|---|
| Sensor/data adapter | Read vehicle counts, occupancy, queue estimates, speed and timestamps |
| State builder | Align readings and build an approach-level state vector |
| Local intersection agent | Generate legal candidate phases and estimate service effect |
| Neighbour coordinator | Receive/send queue and downstream-capacity summaries |
| Utility evaluator | Score valid actions against delay, queue, fairness and spillback objectives |
| Signal executor | Enforce transition timing and communicate phase commands to controller |
| History/learning | Store outcomes and optionally tune policy parameters offline |
| Monitoring dashboard | Display phase, queues, alerts, communication and decision explanations |

## 5.2 Data flow

Sensors feed the state builder. The agent then receives the latest neighbour summaries, removes invalid actions, computes utility values and selects a phase. The signal executor applies the command only when transition rules permit. After the action, the agent observes the change in queue and delay and records the outcome for evaluation or later learning.

# 6. Agent State, Actions and Communication

## 6.1 State representation

For intersection i at decision time t, define the local state as:

```
s_i(t) = {q_a, d_a, w_a, f_a, p_i, e_i, n_i, c_i, z_i}
```

where q is queue length, d is density/occupancy, w is waiting time, f is observed arrival flow, p is current phase, e is elapsed green, n summarises neighbour conditions, c contains pedestrian/emergency calls and z records sensor/message quality. Each approach a has its own queue and waiting-time values.

## 6.2 Action representation

| Action field | Meaning |
|---|---|
| Phase | Which compatible movement group receives green |
| Green duration | Requested extension or duration within configured limits |
| Next phase | Legal successor under the signal plan |
| Coordination intent | Hold, release a platoon, or avoid feeding a blocked downstream link |
| Reason code | Queue pressure, maximum wait, high flow, downstream constraint or fallback |

## 6.3 Neighbour communication

Agents exchange summaries rather than every vehicle record. A message can include intersection ID, outgoing link, timestamp, estimated queue, occupancy, average speed, receiving capacity, current phase and confidence. Messages should have a short validity window; an old queue estimate must not be treated as current truth.

```
{
  "intersection_id": "J2",
  "link_to": "J3",
  "timestamp": "2026-10-10T10:30:05",
  "queue_vehicles": 22,
  "occupancy": 0.78,
  "mean_speed_kmh": 9,
  "receiving_capacity": "LOW",
  "confidence": 0.91
}
```

*The values above are illustrative message fields, not a live city feed.*

# 7. Utility Function and Decision Policy

The utility function makes the trade-offs visible. A high utility means an action is preferred, provided it passes all safety and legal checks. Queue length and waiting time should both matter: queue length represents storage pressure, while waiting time represents service fairness and accumulated delay.

## 7.1 Illustrative utility score

For candidate phase p, define normalized values between 0 and 1 for queue pressure Q(p), waiting pressure W(p), flow served F(p), downstream congestion penalty D(p), and fairness benefit A(p). An example score is:

$$U(p) = 0.30Q(p) + 0.25W(p) + 0.20F(p) + 0.15A(p) - 0.25D(p)$$

The terms estimate the benefit of serving approaches compatible with phase p and the cost of sending vehicles toward a blocked downstream link. The weights are demonstration values only; they must be tuned using simulation and traffic-engineering review. All terms should be scaled consistently before combining them.

## 7.2 Hard constraints before scoring

- Do not switch directly between incompatible greens; complete required yellow and all-red intervals.
- Respect minimum and maximum green time and pedestrian clearance.
- Apply emergency pre-emption only through an approved policy.
- Reject an action that would violate controller state or an approved safety interlock.

## 7.3 Fairness and starvation prevention

An approach with a long maximum wait should receive a growing priority bonus or trigger a maximum-wait rule. The fairness term prevents a high-volume approach from winning every cycle. Safety constraints are not part of the weighted score: they are filters that every action must pass.

## 7.4 Learning extension

An optional learning agent can update parameters from measured delay, queue growth, stops and fairness. For an initial project, use offline training in simulation and freeze a reviewed policy before testing. Online exploration on live roads is inappropriate without a rigorous safety case.

# 9. Proposed Algorithm and Pseudocode

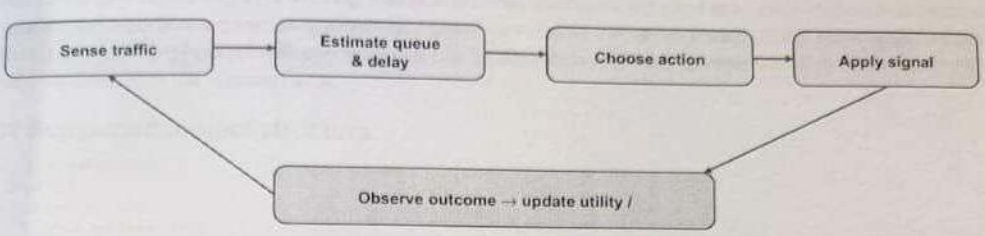



*Figure 3. Agent decision cycle from sensing through evaluation and feedback.*

## 9.1 Algorithm steps

- Read the latest approach-level traffic state and signal-controller status.
- Check sensor quality and obtain valid neighbour summaries.
- Generate candidate phases and durations allowed by the signal plan.
- Discard candidates that violate clearance, pedestrian or controller constraints.
- Estimate queue reduction, waiting-time benefit, flow served, fairness and downstream penalty.
- Calculate utility for every remaining candidate and apply maximum-wait safeguards.
- Choose the highest-utility action; if confidence is poor, use the approved fallback plan.
- Execute the phase transition through the controller, then log the decision and outcome.
- Repeat at the next control interval and update estimates as new observations arrive.

## 9.2 Pseudocode

```
FUNCTION decide(intersection_state, neighbour_messages):
    state, quality = validate_and_build_state(intersection_state)
    neighbours = validate_fresh_messages(neighbour_messages)
    candidates = generate_legal_phase_actions(state)

    best_action = SAFE_FALLBACK
    best_utility = -INFINITY

    FOR action IN candidates:
        IF NOT passes_signal_safety_constraints(action, state):
            CONTINUE

        Q = queue_pressure_served(action, state)
        W = waiting_pressure_served(action, state)
        F = expected_flow_served(action, state)
        A = fairness_benefit(action, state)
        D = downstream_congestion_penalty(action, neighbours)

        utility = 0.30*Q + 0.25*W + 0.20*F + 0.15*A - 0.25*D
        utility = apply_max_wait_override(utility, action, state)

        IF utility > best_utility:
            best_utility = utility
            best_action = action

    execute_through_safe_controller(best_action)
    publish_state_summary()
    log(state, neighbours, best_action, best_utility, quality)
    RETURN best_action
```

The pseudocode is intentionally controller-agnostic. A real implementation must use the signal plan's permitted phases, certified transition logic and exact controller interface.

# 10. Implementation Design

A small prototype should separate sensing, decision-making, communication and execution. Python is suitable for simulation and algorithm experiments; a web dashboard can display the network state. SUMO (Simulation of Urban MObility) is a commonly used traffic simulator that can be considered for testing, while the controller adapter should remain separate from the research logic.

## 10.1 Suggested project structure

```
traffic-agents/
  config/
    network.json
    signal_phases.json
  agents/
    intersection_agent.py
    utility.py
    coordinator.py
    safety_filter.py
  communication/
    message_schema.py
    neighbour_bus.py
  simulation/
    run_scenario.py
    demand_profiles.py
  evaluation/
    metrics.py
    compare_baselines.py
  dashboard/
    app.py
  tests/
    test_safety_filter.py
    test_stale_messages.py
    test_fairness.py
```

## 10.2 Module responsibilities

| Module | Key functions |
|---|---|
| intersection_agent.py | build_state(), generate_actions(), decide(), log_decision() |
| utility.py | queue_benefit(), wait_benefit(), flow_benefit(), downstream_penalty() |
| coordinator.py | publish_summary(), validate_message(), estimate_receiving_capacity() |
| safety_filter.py | is_legal_phase(), clearance_complete(), enforce_min_max_green() |
| metrics.py | mean_delay(), max_queue(), throughput(), fairness_index(), spillback_time() |
| run_scenario.py | load_network(), run_demand(), collect_traces(), export_results() |

## 10.3 Suggested development sequence

Begin with a single four-way intersection and deterministic synthetic arrivals. Add two connected intersections to test downstream penalties. Next, compare the agent with fixed-time and actuated baselines. Add noisy or missing sensor data, delayed messages and emergency/pedestrian scenarios only after basic behaviour is repeatable.

# 11. Worked Example and Sample Input

This fictional example shows how an agent can compare candidate phases. The intersection has four approaches; north-south traffic is queued heavily, while east-west traffic has a smaller queue. The downstream road from the north-south movement is congested, so releasing that movement carries a penalty. Values are illustrative and are not measured results.

## 11.1 Sample intersection state

| Approach | Queue (vehicles) | Max wait (s) | Flow (veh/min) | Downstream condition |
|---|---|---|---|---|
| North | 18 | 72 | 12 | High congestion |
| South | 14 | 58 | 10 | Moderate congestion |
| East | 7 | 31 | 6 | Low congestion |
| West | 11 | 46 | 8 | Low congestion |

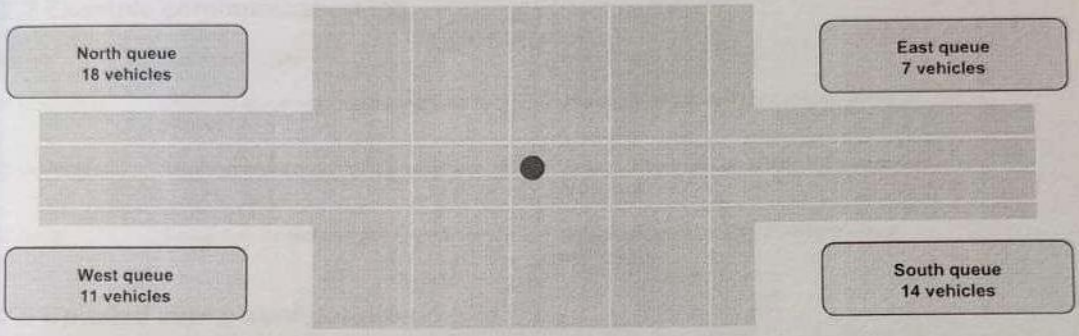



*Figure 4. Illustrative intersection layout and approach queues.*

## 11.2 Candidate phases

| Candidate phase | Queue benefit Q | Wait benefit W | Flow F | Fairness A | Downstream penalty D |
|---|---|---|---|---|---|
| North-South green | 0.90 | 0.85 | 0.90 | 0.60 | 0.85 |
| East-West green | 0.55 | 0.50 | 0.55 | 0.80 | 0.10 |

## 11.3 Calculation

North-South: $U = 0.30(0.90) + 0.25(0.85) + 0.20(0.90) + 0.15(0.60) - 0.25(0.85) = 0.4925$.

East-West: $U = 0.30(0.55) + 0.25(0.50) + 0.20(0.55) + 0.15(0.80) - 0.25(0.10) = 0.4925$.

The scores are tied under these illustrative values. The agent should apply a defined tie-breaker, such as serving the approach with the greater maximum wait, comparing projected queue growth, or selecting the action that reduces spillback risk. It should not claim that one phase is clearly superior from a tie.

# 12. Sample Output and Interpretation

## 12.1 Example agent output

```
Intersection: J2
Decision timestamp: 2026-10-10 10:30:05 (illustrative)
Candidate actions evaluated: 2
North-South utility: 0.4925
East-West utility:   0.4925
Tie-break check: maximum waiting time and downstream storage
Selected action: East-West green (illustrative tie-break outcome)
Requested green: 24 seconds (within configured limits)
Reason: North-South downstream congestion is high; East-West
        has a lower receiving-link penalty and waiting time is lower.
Safety checks: passed in the illustrative scenario
Neighbour messages used: J1 -> J2, J3 -> J2
Decision status: simulated / not connected to a live controller
```

The selected phase above is an example of a policy-defined tie-break, not a mathematically unique result. A real system would calculate utilities from live or simulated state and apply explicit rules for ties, maximum waiting time and safety.

## 12.2 Example communication log

| Sender | Message summary | Agent response |
|---|---|---|
| J1 (upstream) | Outbound queue high; release estimate 15 vehicles | J2 anticipates a platoon and checks storage |
| J3 (downstream) | Receiving capacity low; queue occupancy 78% | J2 applies downstream penalty to movements feeding J3 |
| J2 (local) | Selected phase and expected release window | Neighbour agents update short-term arrival estimates |

## 12.3 Updated signal configuration

| *Field* | **Illustrative value** |
|---|---|
| Current phase | East-West green |
| Green duration | 24 s, subject to the approved phase plan |
| Next transition | Yellow, then required all-red clearance |
| Monitoring trigger | Re-evaluate if queue or waiting time changes materially |
| Fallback | Approved local fixed-time/actuated plan if sensors or messages fail |

## 12.4 Interpretation

The sample demonstrates the required outputs: a selected signal timing, an agent decision, neighbour communication and an updated configuration. The utility score is a ranking aid, not a guarantee of lower real-world delay. Performance must be established experimentally against baseline controllers.

# 13. Testing and Evaluation Plan

Testing should begin in simulation with repeatable traffic demand, then expand to varied demand and failure cases. Comparisons must use the same road network, demand profile, warm-up period and evaluation interval. Results should be reported over multiple random seeds rather than one favourable run.

## 13.1 Functional test cases

| Test | Scenario | Expected behaviour |
|---|---|---|
| T1 | Balanced traffic on all approaches | Choose a reasonable phase and avoid unnecessary extensions |
| T2 | One approach has very long waiting time | Increase service priority within legal limits |
| T3 | Downstream road is nearly full | Penalise or delay feeding movement when safe alternatives exist |
| T4 | Neighbour message is stale | Ignore or down-weight stale data and continue safely |
| T5 | One detector fails | Flag low confidence and use approved fallback/estimation |
| T6 | Pedestrian call is active | Respect pedestrian service and clearance requirements |
| T7 | Emergency vehicle request | Apply only the approved emergency pre-emption policy |
| T8 | Communication link is lost | Continue local safe control and log degraded coordination |
| T9 | Rapid demand surge | Recompute action and ensure queues do not starve other approaches |
| T10 | Conflicting candidate phases | Reject combinations that violate signal conflict rules |

## 13.2 Evaluation metrics

- Mean control delay per vehicle and total delay across the network.
- Mean and maximum queue length; frequency and duration of spillback.
- Throughput, travel time, stops per vehicle and average speed.
- Fairness: maximum waiting time or dispersion in approach waiting times.
- Safety/operations: invalid phase transitions, fallback frequency and controller faults.
- Coordination: stale-message rate, communication overhead and sensitivity to link failure.

## 13.3 Baselines and experimental design

Compare against fixed-time signals, actuated local control and (if available) a coordinated timing plan. Use low, medium, peak and incident demand. Tune utility weights on separate scenarios and report results on held-out scenarios to reduce overfitting. A successful prototype should demonstrate repeatable improvements without safety-rule violations; numerical improvement must not be claimed until measured.

# 14. Safety, Limitations and Deployment

Traffic signals are safety-critical infrastructure. An AI policy may recommend an action, but a trusted controller and independent interlocks must enforce conflict-free phase transitions. The system should be tested under normal demand, detector failures, communication outages, unusual traffic patterns and restart conditions before any real-world trial.

## 14.1 Safety principles

- Hard safety rules override utility and learned policies.
- No direct transition between incompatible green movements; use configured yellow and all-red intervals.
- Respect pedestrian calls, accessible crossing time and emergency policies.
- Use conservative fallback timing when sensor confidence is poor.
- Log all overrides and provide a manual operator override.
- Secure communications and restrict commands to authenticated controllers.

## 14.2 Uncertainty and failure handling

| Limitation / failure | Mitigation |
|---|---|
| Noisy vehicle counts | Smooth estimates, compare redundant sensors and expose confidence |
| Stale neighbour state | Timestamp messages, set expiry and down-weight uncertain values |
| Utility weights are hand-selected | Tune in simulation; validate on unseen demand profiles |
| Unexpected incidents | Detect abnormal queue growth and fall back or request operator review |
| Agent oscillation | Use hysteresis, minimum hold times and bounded changes |
| Learning policy behaves unexpectedly | Train offline, constrain actions and retain a reviewed fallback |
| Network-wide interactions are complex | Evaluate multi-intersection scenarios and spillback explicitly |

## 14.3 Deployment stages

Stage 1: offline algorithm tests. Stage 2: microscopic simulation on a small grid. Stage 3: hardware/controller-in-the-loop tests. Stage 4: shadow mode, where recommendations are logged but not applied. Stage 5: limited, authorised pilot with traffic-engineer supervision and rollback capability. Progression requires evidence and formal approval at each stage.

## 14.4 Limitations

The illustrative utility is not calibrated, the sample values are synthetic and no field performance is claimed. The report does not specify a city-specific road graph, legal signal timings, pedestrian standards or an emergency-service interface. These must be supplied by the responsible transport authority.

# 15. Conclusion and References

## 15.1 Conclusion

This report proposed an AI-based multi-agent traffic management framework in which every intersection observes local traffic, evaluates legal signal actions and communicates with neighbouring intersections. A hybrid design combines reactive safety rules, goal-driven queue and delay objectives, utility-based action ranking and an optional learning component. Downstream congestion penalties help prevent an upstream agent from releasing traffic into a blocked link, while fairness and maximum-wait logic reduce the risk of neglecting low-volume approaches.

The worked example demonstrated the expected system outputs: a selected signal phase and duration, an explanation of the local decision, inter-agent messages and an updated signal configuration. It also showed that illustrative utility scores can tie, requiring an explicit tie-break rule. The next stage is to implement the modular prototype, simulate connected intersections, compare against fixed-time and actuated baselines, tune parameters on training scenarios and evaluate on unseen traffic conditions.

The key design principle is that intelligence must remain subordinate to safety. Real deployment requires approved signal plans, robust controllers, careful treatment of sensor uncertainty, cybersecurity, operator oversight and staged field validation. The report therefore presents a feasible academic design rather than a validated operational traffic-control system.

## 15.2 References


[1] S. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed., Pearson, 2021. Background on intelligent agents, search and utility-based decision-making.

[2] M. Wooldridge, *An Introduction to MultiAgent Systems*, 2nd ed., Wiley, 2009. Foundations of autonomous agents and multi-agent coordination.

[3] M. J. Smith, "Traffic signal control and traffic management" — general background topic; consult relevant transportation engineering texts and local signal standards for implementation-specific rules.

[4] Eclipse SUMO, "Simulation of Urban MObility" documentation. https://sumo.dlr.de/docs/

[5] Federal Highway Administration, *Traffic Signal Timing Manual*, FHWA-HOP-08-024, 2008. https://ops.fhwa.dot.gov/publications/fhwahop08024/

[6] National Transportation Communications for ITS Protocol (NTCIP), traffic signal controller and transportation management communication standards. https://www.ntcip.org/


Appendix note: all sample queues, utilities, messages and green durations in this report are illustrative. Replace them with documented network data and approved signal constraints before any practical use.